

WIPRO PYTHON AUTOMATION COURSE

MODULE_3_CODE_REPORT

PYTHON BDD RESTFUL AUTOMATIONS

NAME: - Amit Raj

Course: - Python Automation Testing

Technology: - Python, BDD, REST APIs

Enrolment No.: - 12023052018002

Stream: - CSBS

Semester: - 7th

Date: - 18/06/2026

Folder name: -

pages/login_page.py

Code: -

```
from selenium.webdriver.common.by import By

class LoginPage:

    USERNAME = (By.ID, "user-name")
    PASSWORD = (By.ID, "password")
    LOGIN_BUTTON = (By.ID, "login-button")
    ERROR_MESSAGE = (By.CSS_SELECTOR, "[data-test='error']")

    def __init__(self, driver):
        self.driver = driver

    def enter_username(self, username):
        self.driver.find_element(*self.USERNAME).clear()
        self.driver.find_element(*self.USERNAME).send_keys(username)

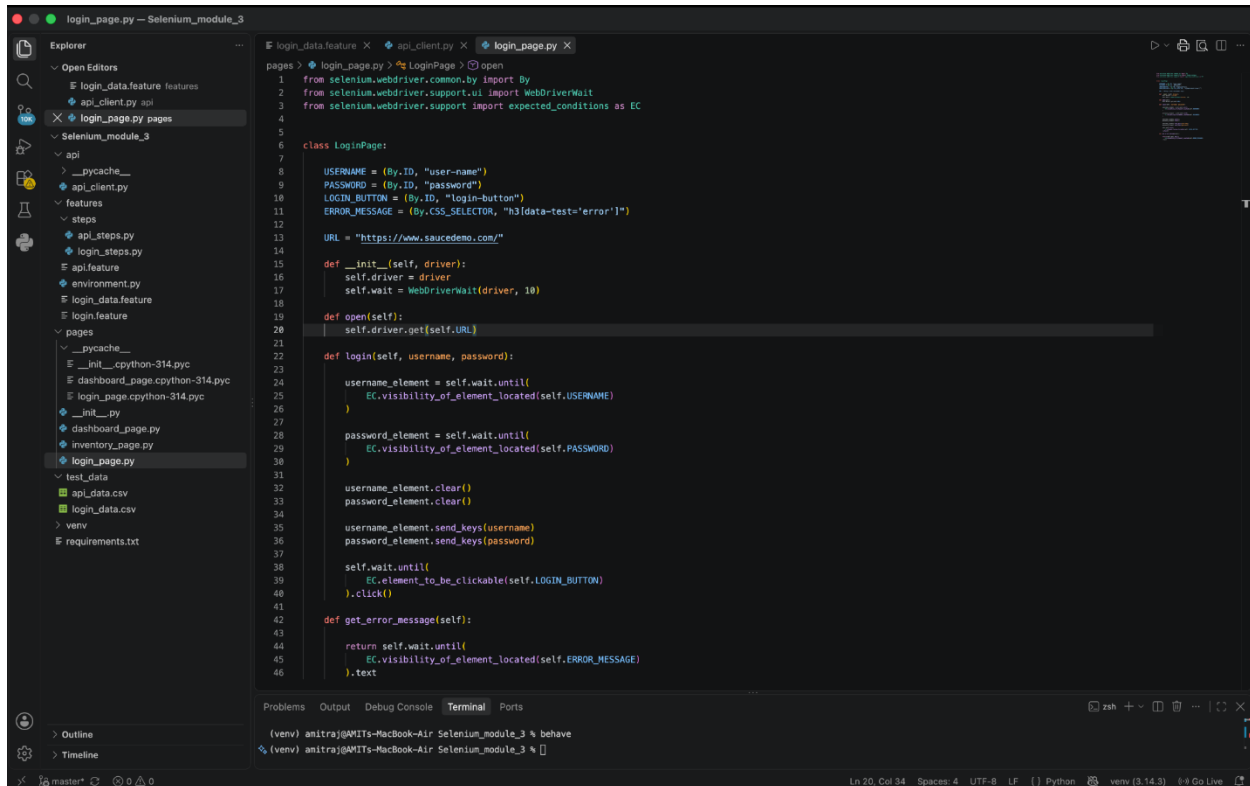
    def enter_password(self, password):
        self.driver.find_element(*self.PASSWORD).clear()
        self.driver.find_element(*self.PASSWORD).send_keys(password)

    def click_login(self):
        self.driver.find_element(*self.LOGIN_BUTTON).click()

    def login(self, username, password):
        self.enter_username(username)
        self.enter_password(password)
        self.click_login()

    def get_error_message(self):
        return self.driver.find_element(*self.ERROR_MESSAGE).text
```

Screenshots: -



Folder name: -

features/login.feature

Code: -

Feature: SauceDemo Login

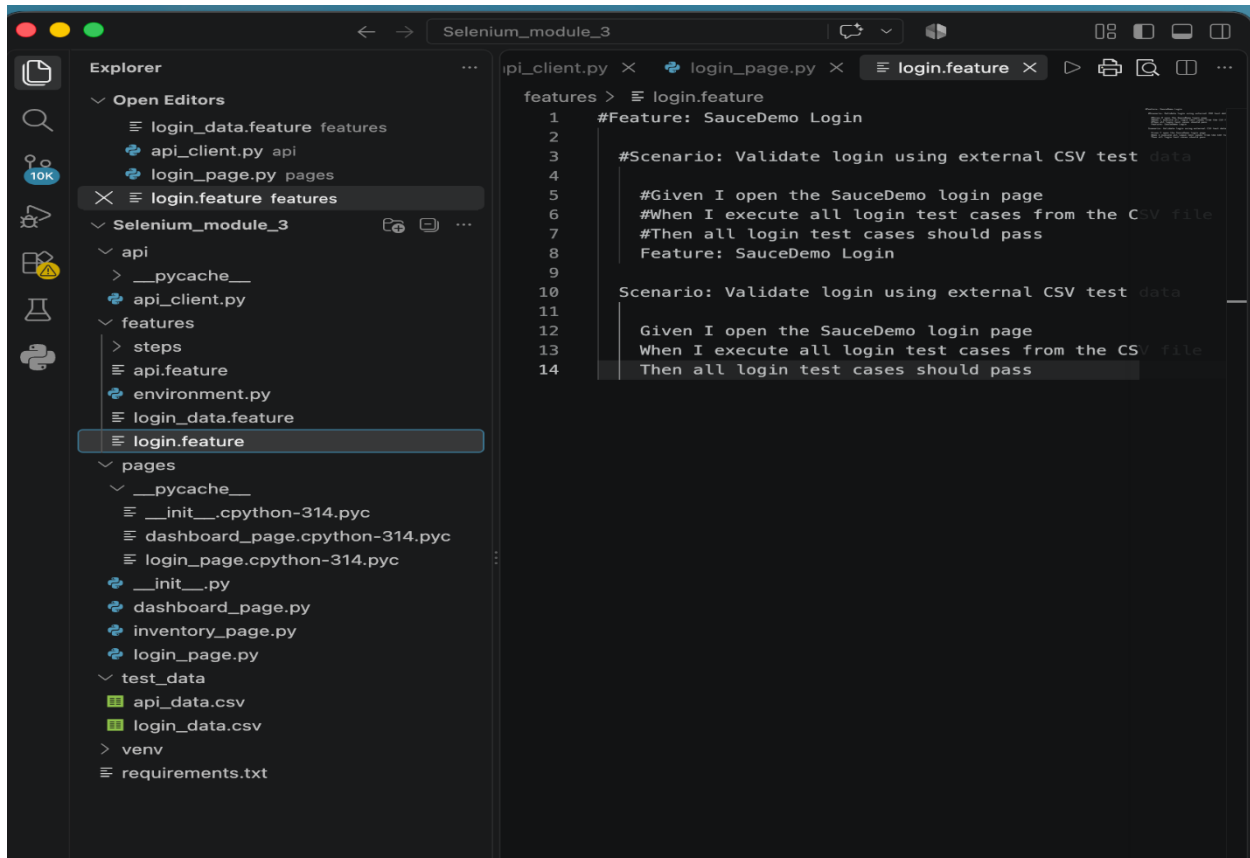
Scenario: Successful login with valid credentials

Given I open the SauceDemo login page

When I login with username "standard_user" and password "secret_sauce"

Then I should be redirected to the inventory page

Screenshots: -



Folder Name: -

pages/dashboard_page.py

Code: -

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class DashboardPage:

    INVENTORY_CONTAINER = (

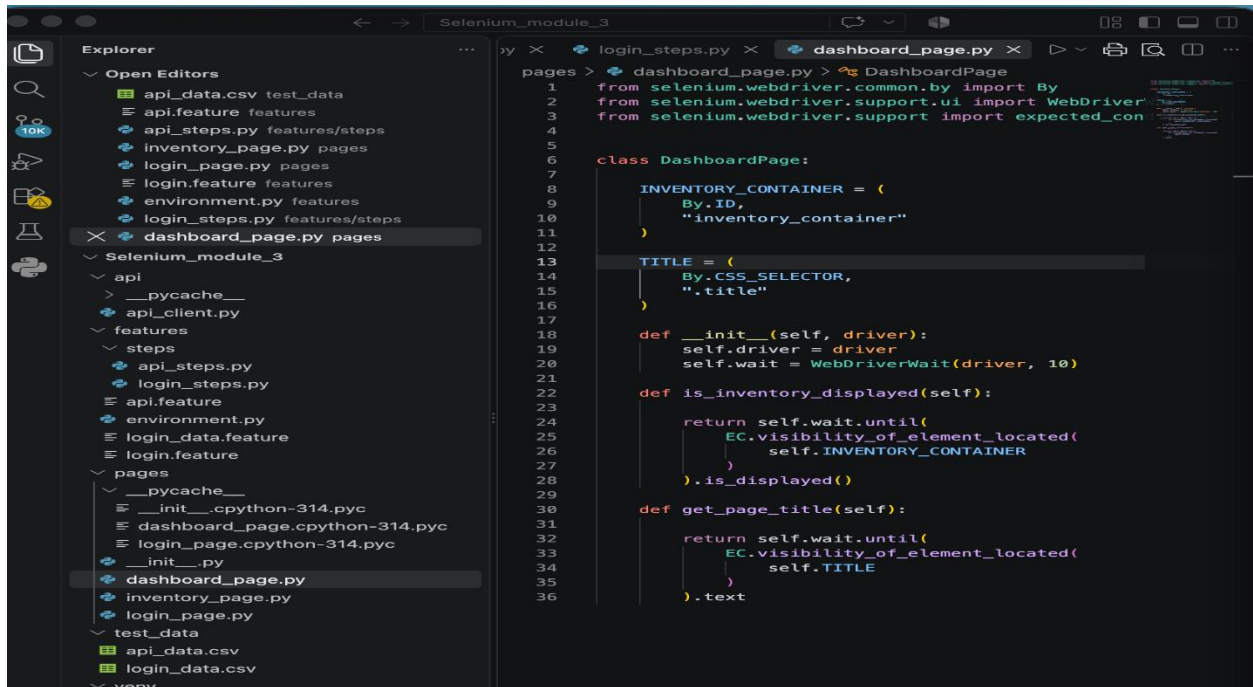
        By.ID,

        "inventory_container"

    )
```

```
TITLE = (  
    By.CSS_SELECTOR,  
    ".title"  
)  
  
def __init__(self, driver):  
    self.driver = driver  
    self.wait = WebDriverWait(driver, 10)  
  
def is_inventory_displayed(self):  
    return self.wait.until(  
        EC.visibility_of_element_located(  
            self.INVENTORY_CONTAINER  
        )  
    ).is_displayed()  
  
def get_page_title(self):  
    return self.wait.until(  
        EC.visibility_of_element_located(  
            self.TITLE  
        )  
    ).text
```

Screenshots: -



Folder Name: -

features/environment.py

Code: -

```
from selenium import webdriver

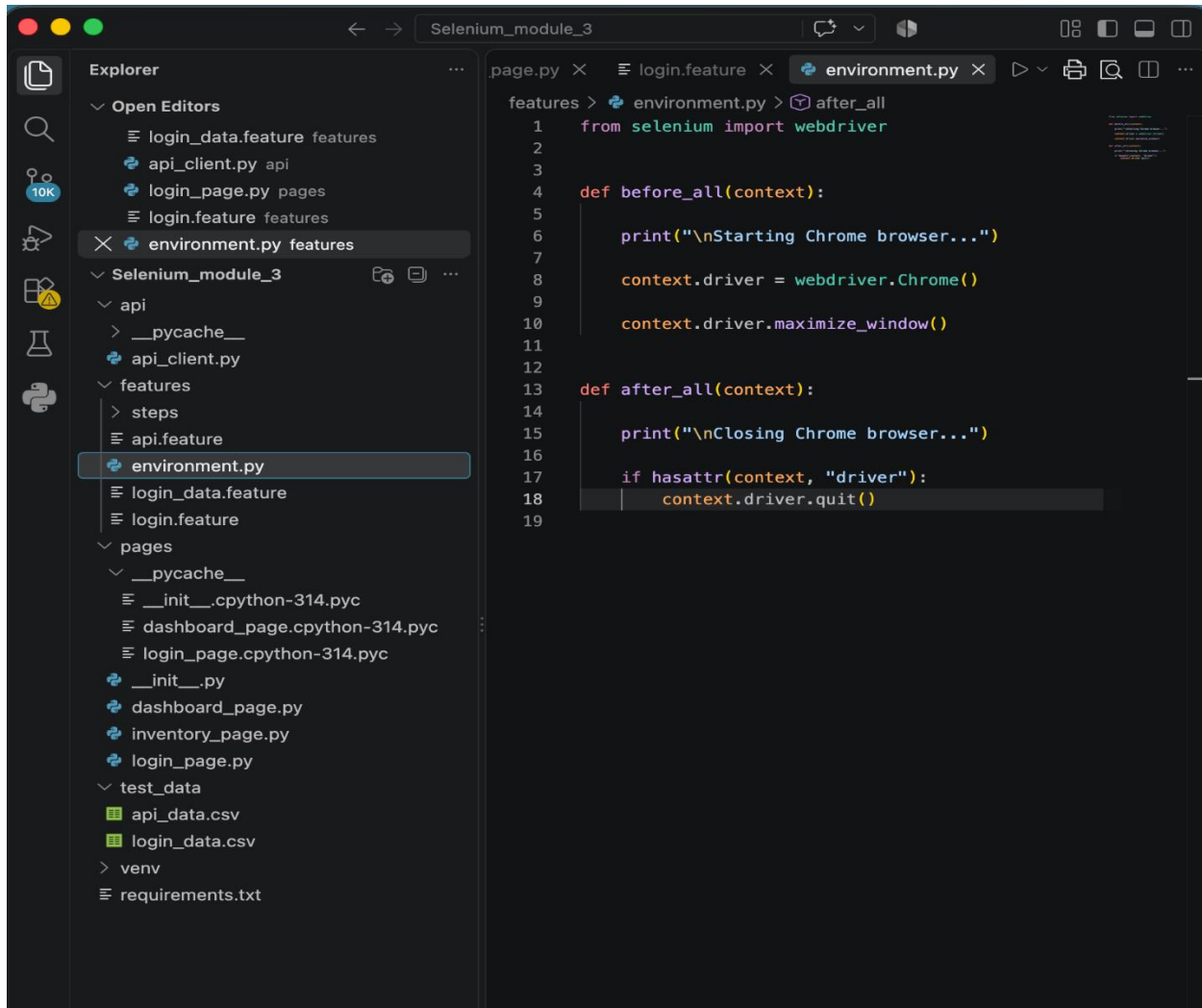
def before_scenario(context, scenario):

    context.driver = webdriver.Chrome()
    context.driver.maximize_window()

def after_scenario(context, scenario):

    context.driver.quit()
```

Screenshots: -



Folder Name: -

features/steps/login_steps.py

Code: -

```
import os

import pandas as pd

from behave import given, when, then

from pages.login_page import LoginPage

from pages.dashboard_page import DashboardPage
```

```

@given("I open the SauceDemo login page")
def step_open_login_page(context):
    context.login_page = LoginPage(context.driver)
    context.login_page.open()

@when("I execute all login test cases from the CSV file")
def step_execute_login_tests(context):
    csv_path = os.path.join(
        "test_data",
        "login_data.csv"
    )
    test_data = pd.read_csv(
        csv_path,
        keep_default_na=False
    )
    context.results = []
    for _, row in test_data.iterrows():
        test_case = row["test_case"]
        username = row["username"]
        password = row["password"]
        expected_result = row["expected_result"]
        expected_error = row["expected_error"]
        print(f"\nRunning {test_case}")
        # Open login page before every test case
        context.login_page.open()
        # Perform login
        context.login_page.login(

```

```

        username,
        password
    )
    if expected_result == "success":
        dashboard_page = DashboardPage(
            context.driver
        )
        current_url = context.driver.current_url
        inventory_displayed = (
            dashboard_page.is_inventory_displayed()
        )
        page_title = (
            dashboard_page.get_page_title()
        )
        passed = (
            "/inventory.html" in current_url
            and inventory_displayed
            and page_title == "Products"
        )
        actual_result = (
            f"URL={current_url}, "
            f"Title={page_title}"
        )
    else:
        actual_error = (
            context.login_page.get_error_message()

```

```

    )

    passed = actual_error == expected_error

    actual_result = actual_error
context.results.append(
    {
        "test_case": test_case,
        "passed": passed,
        "actual": actual_result,
        "expected": (
            "Success"
            if expected_result == "success"
            else expected_error
        )
    }
)

print(
    f"{test_case}: "
    f"{'PASSED' if passed else 'FAILED'}"
)

@then("all login test cases should pass")
def step_validate_all_tests(context):
    failed_tests = [
        result
        for result in context.results
        if not result["passed"]
    ]

```

```
for result in context.results:

    print(
        f"\n{result['test_case']}: "
        f"'{PASSED}' if result['passed'] else 'FAILED'"
    )

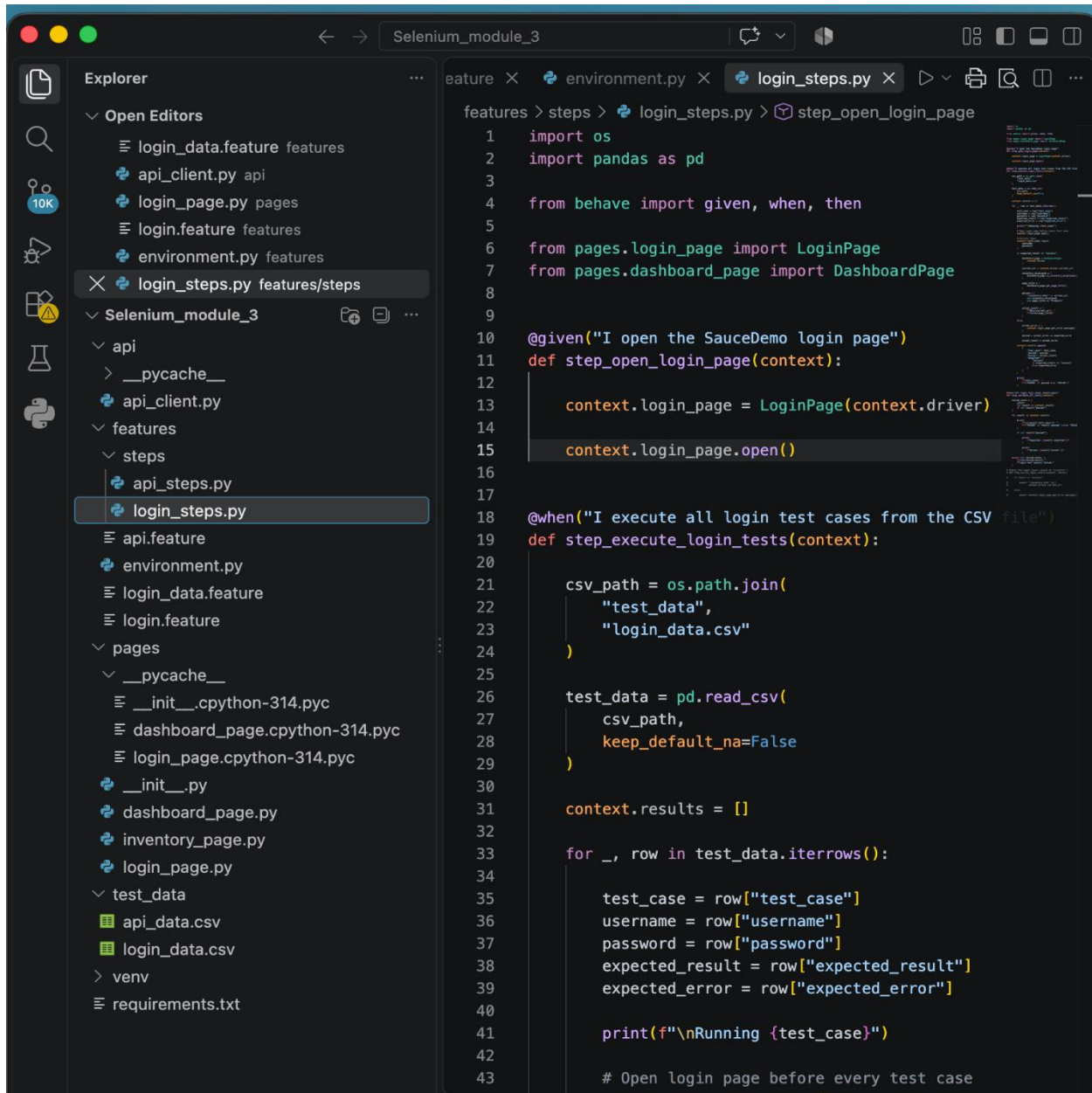
    if not result["passed"]:

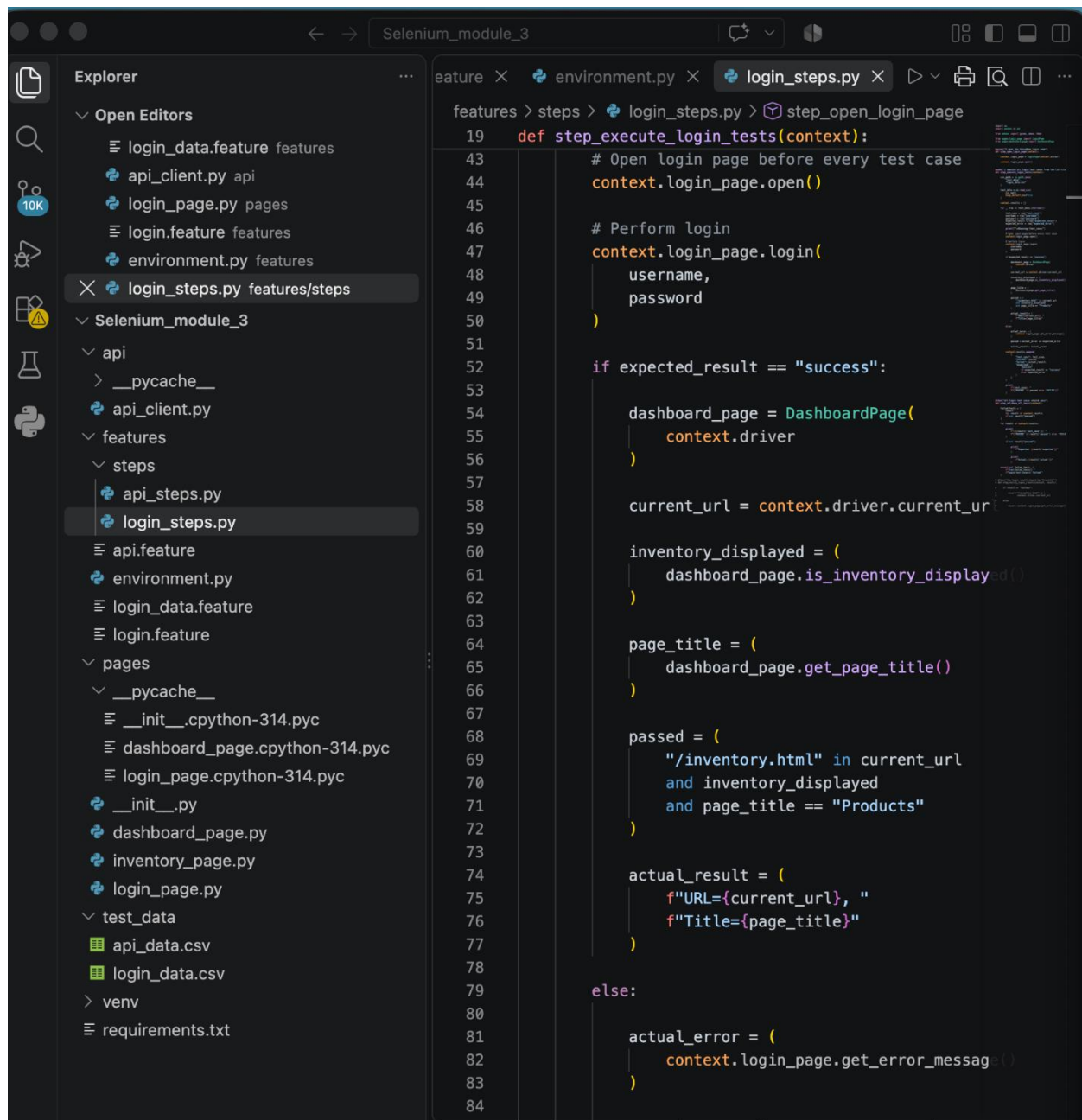
        print(
            f"Expected: {result['expected']}"
        )

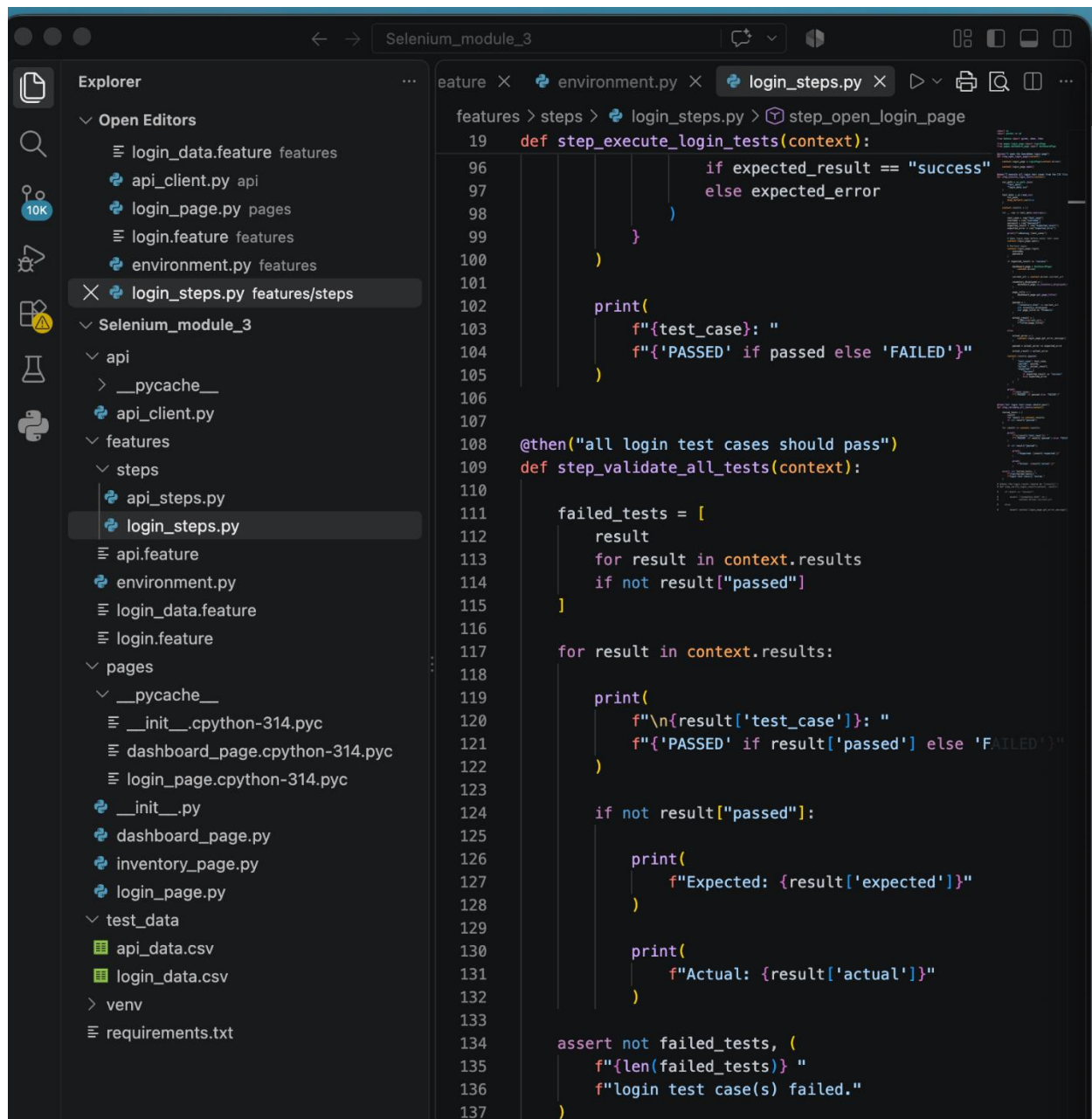
        print(
            f"Actual: {result['actual']}"
        )

    assert not failed_tests, (
        f"{len(failed_tests)} "
        f"login test case(s) failed."
    )
```

Screenshots: -







Folder name: -

api/api_client.py

Code:-

```
import requests

class APIClient:

    BASE_URL = "https://jsonplaceholder.typicode.com"

    def get_post(self, post_id):

        return requests.get(

            f"{self.BASE_URL}/posts/{post_id}"

        )

    def create_post(self, payload):

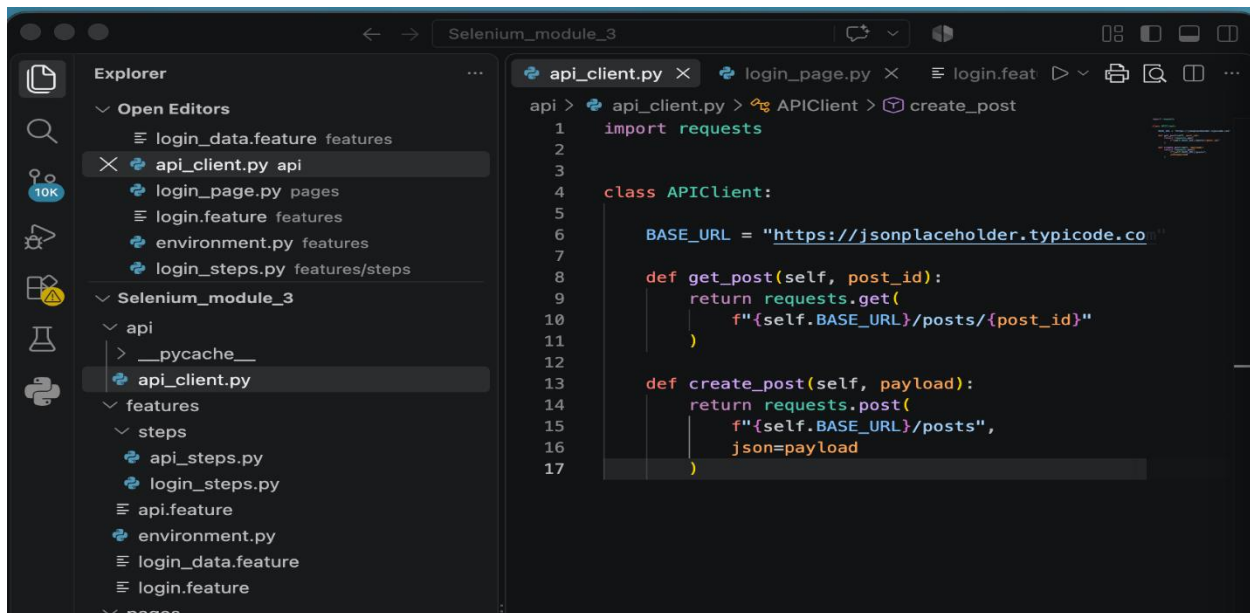
        return requests.post(

            f"{self.BASE_URL}/posts",

            json=payload

        )
```

Screenshots: -



Folder Name: -

test_data/api_data.csv

Code: -

test_case,post_id,expected_status

TC001,1,200

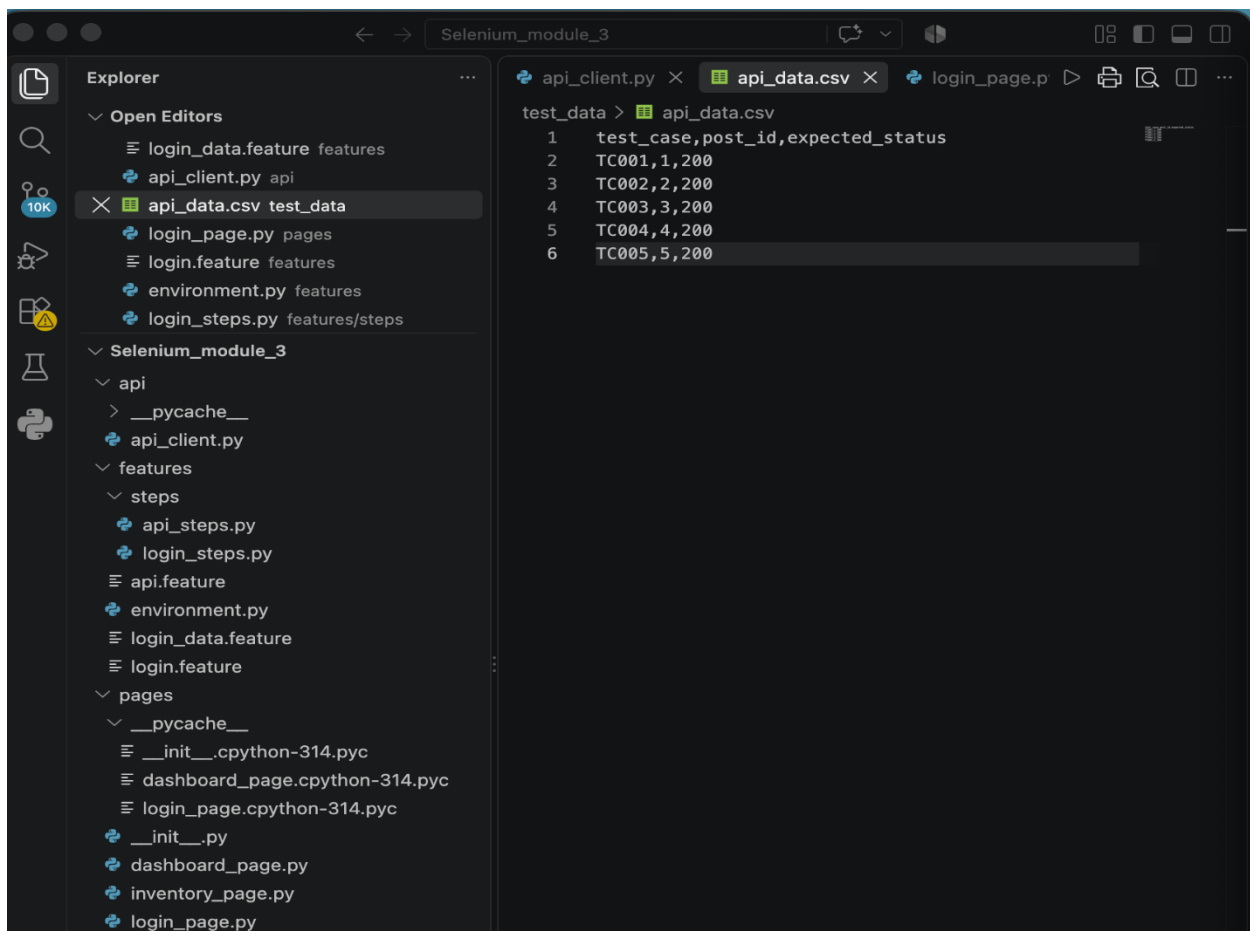
TC002,2,200

TC003,3,200

TC004,4,200

TC005,5,200

Screenshots: -



Folder Name: -

features/api.feature

Code: -

Feature: API Automation

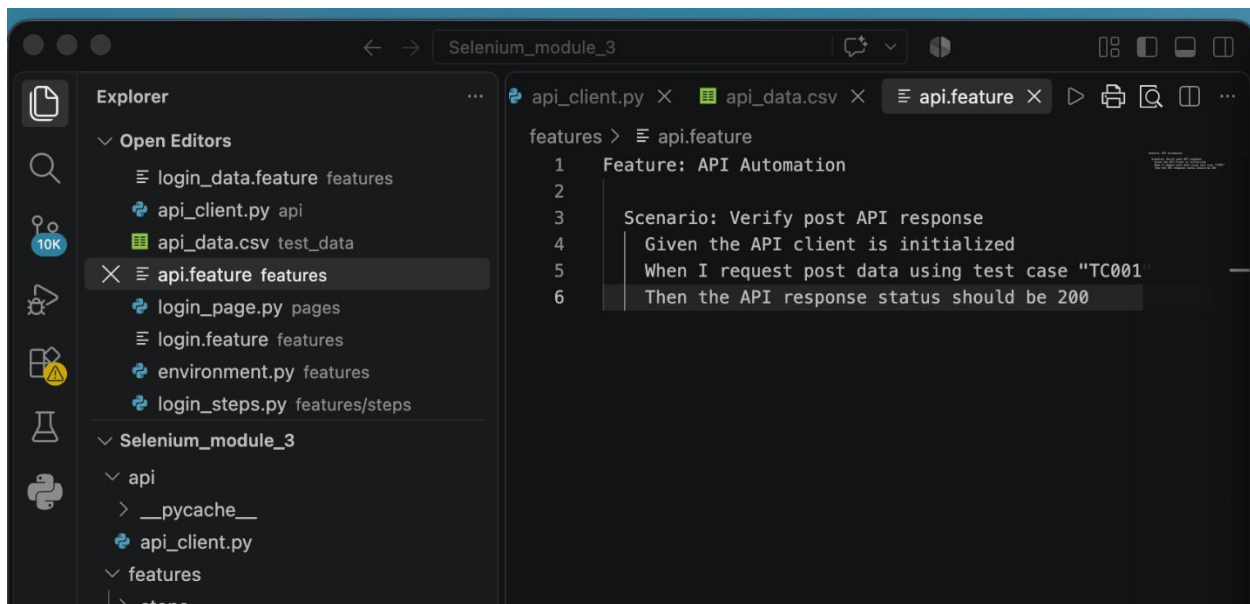
Scenario: Verify post API response

Given the API client is initialized

When I request post data using test case "TC001"

Then the API response status should be 200

Screenshots: -



Folder name: -

features/steps/api_steps.py

Code: -

```
from behave import given, when, then

import pandas as pd

from api.api_client import APIClient

@given("the API client is initialized")
def step_initialize_api(context):
    context.api_client = APIClient()

    context.test_data = pd.read_csv(
        "test_data/api_data.csv"
    )

    @when("I request post data using test case \"{test_case}\"")
    def step_request_post(context, test_case):
        row = context.test_data[
            context.test_data["test_case"] == test_case
        ].iloc[0]

        post_id = int(row["post_id"])

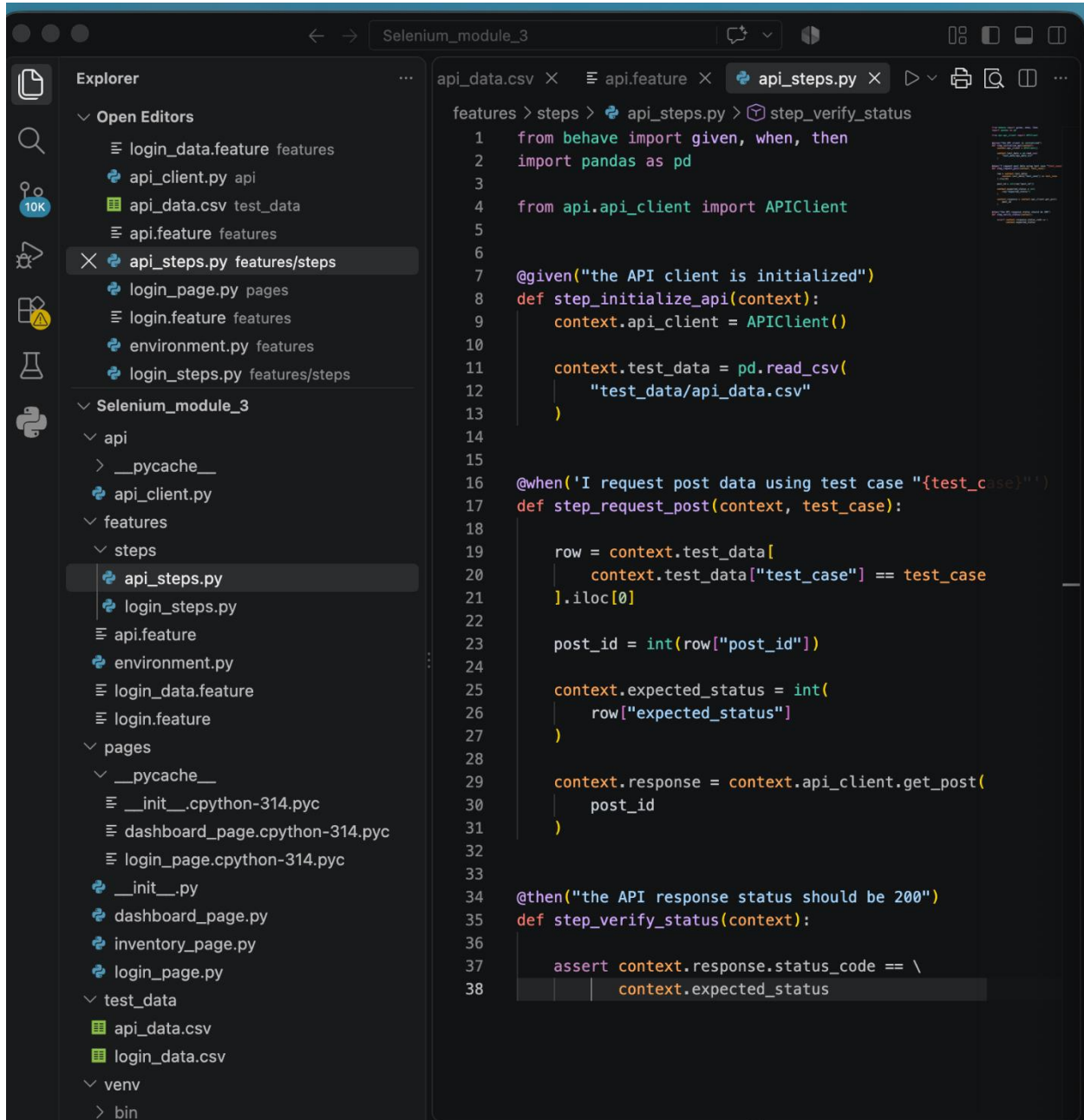
        context.expected_status = int(
            row["expected_status"]
        )

        context.response = context.api_client.get_post(
            post_id
        )

    @then("the API response status should be 200")
    def step_verify_status(context):
```

```
assert context.response.status_code == \
    Context.expected_status
```

Screenshots: -



Folder name:-

pages/inventory_page.py

Code: -

```
from selenium.webdriver.common.by import By
```

```
class InventoryPage:
```

```
    TITLE = (By.CLASS_NAME, "title")
```

```
    INVENTORY_CONTAINER = (By.ID, "inventory_container")
```

```
    def __init__(self, driver):
```

```
        self.driver = driver
```

```
    def get_page_title(self):
```

```
        return self.driver.find_element(
```

```
            *self.TITLE
```

```
        ).text
```

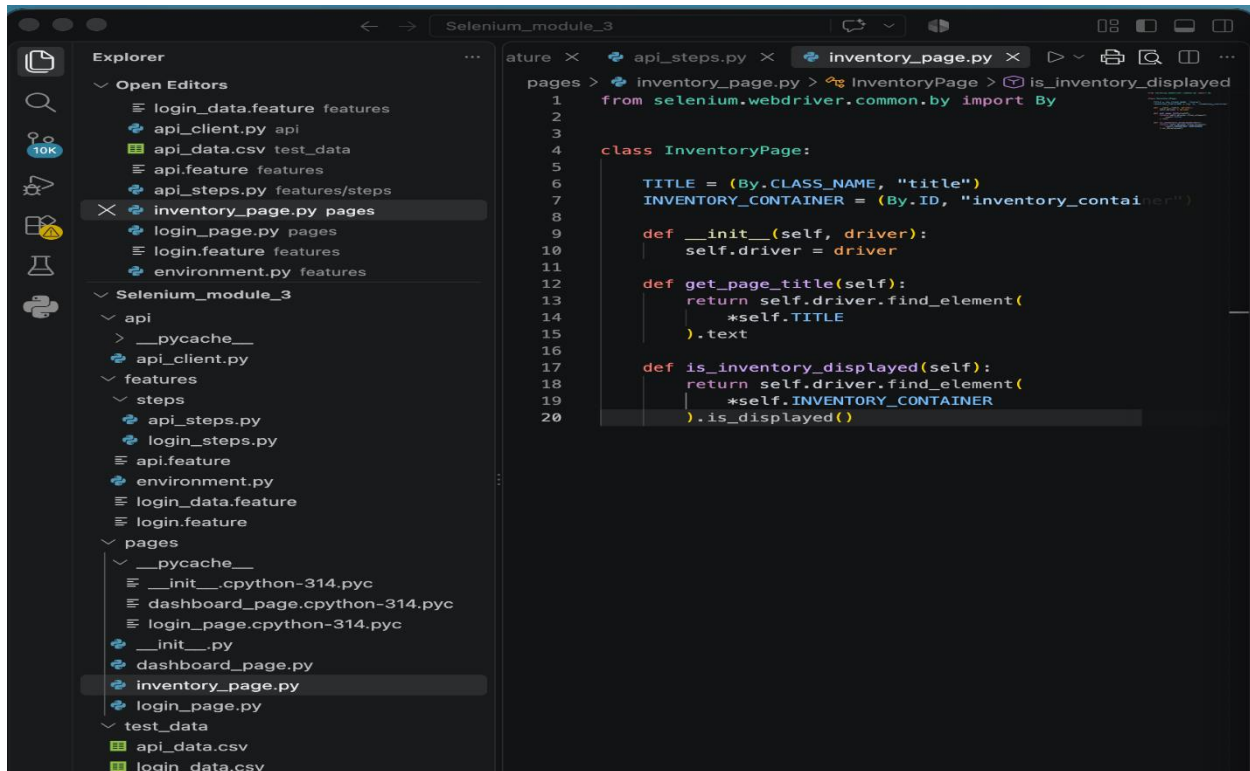
```
    def is_inventory_displayed(self):
```

```
        return self.driver.find_element(
```

```
            *self.INVENTORY_CONTAINER
```

```
        ).is_displayed()
```

Screenshots: -



Folder name: -

features/login_data.feature

Code: -

Feature: Data Driven Login

Scenario Outline: Login using different credentials

Given I open the SauceDemo login page

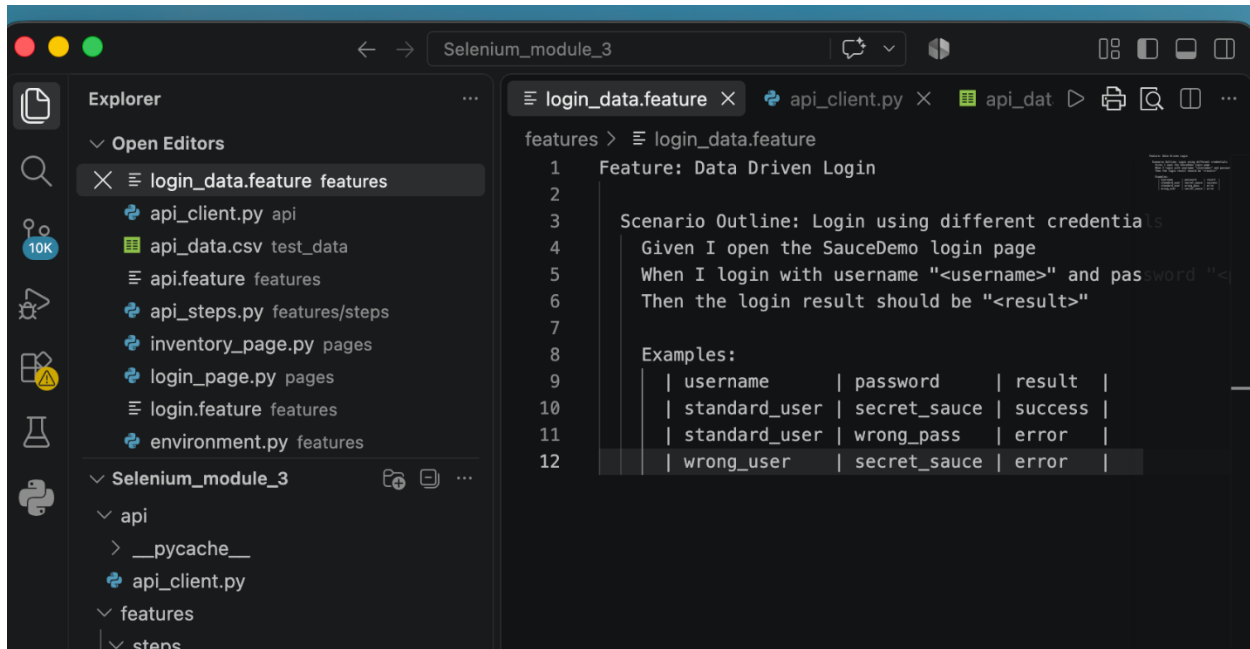
When I login with username "<username>" and password "<password>"

Then the login result should be "<result>"

Examples:

username	password	result
standard_user	secret_sauce	success
standard_user	wrong_pass	error
wrong_user	secret_sauce	error

Screenshots: -



Folder name: -

Requirements.txt

Code: -

selenium

behave

pandas

Requests

Screenshots: -

